

[Mobile] Examen Técnico — Senior React Native Mobile Engineer

Duración: 30 minutos de sustentación.

Modalidad: Entrega en repositorio GitHub **público** + exposición en vivo de la solución. **Contexto:** Prueba inspirada en apps móviles de nivel productivo donde React Native actúa como *shell* nativo, buena parte de la UI se renderiza como micro-apps web dentro de **WebViews**, la comunicación web ↔ nativo ocurre por `postMessage`, y ciertas capacidades se resuelven con **módulos nativos** empaquetados como **librerías reutilizables**.

1. Objetivo

Construirás una feature móvil de nivel productivo que integra **capa nativa** ↔ **WebView**, **estado global**, **un módulo nativo empaquetado como librería** y **pruebas unitarias**, aplicando buenas prácticas de arquitectura (DDD + al menos un patrón de diseño) y uso responsable de IA (skills y agents).

No es necesario terminar el 100% de la feature. Prioriza **criterio de ingeniería**: estructura, decisiones justificadas, pruebas y una buena comunicación de tu solución.

2. El problema (caso de negocio)

"Bolsillo de Ahorro Programado" (*Savings Goal Wallet*)

Se quiere construir una funcionalidad de **metas de ahorro** ("Bolsillos"): el usuario crea metas, les abona dinero y ve su progreso. La particularidad de la arquitectura es que **el detalle de cada meta y el formulario de abono se renderizan como una micro-app web dentro de un WebView**, mientras que la lista de metas, el estado global, la seguridad y las capacidades del dispositivo viven en la capa **nativa** React Native.

Historias de usuario mínimas:

1. **Como usuario** quiero ver un listado de mis metas de ahorro (nombre, monto objetivo, monto acumulado, % de progreso) en una pantalla **nativa**.
2. **Como usuario** quiero abrir el detalle de una meta, que se renderiza dentro de un **WebView**, y poder **abonar** un valor.
3. **Como usuario** quiero que, al confirmar un abono en la web, el **estado global (Redux)** de la app nativa se actualice y el listado refleje el nuevo acumulado **sin recargar**.

4. **Como usuario** quiero recibir una **confirmación local** (a través de un módulo nativo) cuando alcanzo el 100% de una meta.

Regla de comunicación (obligatoria): la web y el nativo se comunican **exclusivamente por** `postMessage` en ambos sentidos:

- **Nativo → Web:** payload inicial de sesión al cargar el WebView (ej. `{ sessionId, goalId, userInfo }`).
- **Web → Nativo:** eventos de dominio (ej. `{ "type": "DEPOSIT_CONFIRMED", "payload": { "goalId": "g-1", "amount": 50000 } }`).

La micro-app web puede simularse con un HTML estático que emita `window.ReactNativeWebView.postMessage(...)`. **No** se requiere backend real: los datos pueden vivir en memoria/mock o en un repositorio simulado.

Alcance sugerido

- **Debe entregarse (core):** HU 1, 2 y 3 con el ciclo `postMessage` completo + pruebas unitarias en las capas evaluadas.
- **Deseable (si alcanza el tiempo):** HU 4 con el módulo nativo real; persistencia del estado.
- Prioriza calidad sobre cantidad. Un core sólido y probado vale más que 4 HUs a medias.

3. Estructura de entrega — 3 componentes (obligatorio)

La solución se compone de **3 partes**. **Preferido: un solo repositorio público (monorepo) con 3 carpetas.

```
1 examen-rn/
2 | web/           # 1. Micro-app web (WebView) – NO evaluada, NO lleva
3 | libreria/      # 2. Librería custom con componente NATIVO – CON tes
4 | mobile/       # 3. Capa mobile / caparazón (entry final) – CON tes
5
```

#	Carpeta	Qué es	Pruebas unitarias	Uso de IA (agents/skills)	¿Se evalúa?
1	web/	Micro-app web que se renderiza	❌ No	Opcional	❌ No se evalúa (salvo que el

		dentro del WebView. Solo debe emitir y recibir postMes sage .			postMes sage funcione)
2	librería/ a/	Librería custom que contiene un compone nte/módul o nativo (Kotlin/Swi ft) y se implemen ta dentro de la capa mobile como dependen cia.	✓ Obligatori as	✓ Obligatori o	✓ Sí
3	mobile/	Capa mobile (caparazón): la app React Native final que consume la librería y hospeda el WebView.	✓ Obligatori as	✓ Obligatori o	✓ Sí

Reglas:

- La **web NO tendrá pruebas unitarias y NO será evaluada**; lo único que exigimos de ella es que la comunicación por `postMessage` funcione con la app mobile.
- La **librería debe ser custom, contener algo nativo y ser consumida por la capa mobile** (no código copiado dentro de la app).
- Todo debe estar en un **repositorio público de GitHub** con historial de commits **incremental y descriptivo** (no un único commit final).

Cada carpeta evaluada (`libreria/` , `mobile/`) debe tener su `README.md` . El **README raíz** debe incluir:

- Cómo instalar y correr (`iOS` / `Android`) y versiones de Node/React Native usadas.
- Cómo correr los tests y ver el coverage de `libreria/` y `mobile/` .
- Diagrama o descripción de la **arquitectura** elegida.
- Catálogo de los `postMessage` soportados (contrato web ↔ nativo).
- Sección **"Uso de IA"** (ver punto 4).

Envía el link del repo antes del inicio de la sustentación.

4. Uso de IA — Skills y Agents (obligatorio documentarlo)

Se **permite y se valora** el uso de asistentes de IA (Claude Code, Copilot, etc.). Lo que evaluamos no es "si usó IA", sino **cómo la gobernó**. El uso de IA (agents/skills) es **obligatorio en las capas `libreria/` y `mobile/`**; en `web/` es opcional.

Debes entregar (carpeta `.claude/` o `docs/ia/` en las capas evaluadas):

1. **Al menos 1 skill propia** que encapsule una tarea repetible de tu flujo. Ejemplos válidos:
 - "Generar un slice de Redux (action + reducer + selector + test) siguiendo la convención del proyecto".
 - "Crear un handler de `postMessage` con su contrato tipado y su test".
2. **Al menos 1 agent/subagente** con un propósito claro (ej. un *reviewer* que verifique cobertura de tests y convenciones, o un *diagnóstico* de errores).
3. Un breve documento `docs/ia/US0_IA.md` explicando:
 - Qué generaste con IA vs. qué escribiste/ajustaste tú.
 - Qué prompts/skills usaste y por qué.
 - Qué **rechazaste o corregiste** de lo que propuso la IA (criterio crítico).

En la sustentación se te pedirá **justificar** cualquier fragmento; "lo generó la IA" no exime de entenderlo.

5. Requisitos técnicos obligatorios

#	Requisito	Detalle
5.1	Versión de React Native / CLI	El proyecto debe crearse con la CLI oficial de la comunidad (<code>@react-native-community/cli</code> , <code>npx @react-native-community/cli init</code>). Expo está prohibido (ni bare ni managed). Usa una versión reciente y estable de React Native (0.81+ con React 19). Ver §5.7 para puntos extra.
5.2	TypeScript	El dominio, los contratos de <code>postMessage</code> , los slices de Redux y la lógica de negocio deben estar tipados . Nada de <code>any</code> sin justificación.
5.3	WebViews + postMessage	El detalle/abono se renderiza en un <code>WebView</code> . Comunicación

		bidireccional por <code>postMessage</code> . Debe existir un contrato explícito de mensajes (tipos discriminados).
5.4	Redux	Estado global con Redux (Redux Toolkit recomendado). El listado de metas y el resultado del abono viven en el store. Los mensajes de la web despachan acciones.
5.5	Librería custom con componente nativo	Construir una librería propia (carpeta <code>libreria/</code>) que contenga un componente/módulo nativo (Android y/o iOS) y se implemente dentro de la capa <code>mobile</code> como dependencia (no código suelto dentro de la app). Ejemplo: <code>rn-savings-notifier</code> que exponga a JS <code>notifyGoalCompleted(goalName: string)</code> y dispare una notificación / <code>Toast</code> / <code>UILocalNotification</code> nativa. Ver §6.
5.6	Pruebas unitarias	Obligatorias en <code>libreria/</code> y

		mobile/ . Jest + React Native Testing Library. Mínimo: reducers/selectors, el parser de <code>postMessage</code> , la API de la librería, y al menos un componente/hook. Coverage razonable del core (indica tu meta, ej. $\geq 70\%$ en la capa de dominio). La capa web/ NO lleva pruebas y no se evalúa.
5.7	Puntos extra por versión	+puntos por usar nuestra versión de React Native (0.81 con React 19) — es la que usamos y necesitamos —, , y por una justificación sólida del patrón de diseño y la arquitectura elegida (trade-offs, alternativas, escalabilidad). El módulo nativo como TurboModule suma más que el bridge clásico (<code>NativeModules</code>), siempre que lo justifiques.

6. Cómo debe ser la librería (`libreria/` — módulo nativo reutilizable)

Este es un eje central del examen: **demostrar que sabes crear una librería de React Native con código nativo**, no solo consumir una.

Andamiaje (obligatorio)

La librería debe construirse usando `react-native-builder-bob` ([callstack/react-native-builder-bob](https://callstack.io/react-native-builder-bob)) de Callstack. Debe contener **código nativo** (no una librería solo-JavaScript).

Requisitos de la librería:

- **Paquete independiente** con su propio `package.json`, versionado y API pública clara (creada con `create-react-native-library` / `react-native-builder-bob`, o publicable vía `npm pack` / registro local `verdaccio`).
- **Código nativo real** en al menos una plataforma (Android en Kotlin/Java y/o iOS en Swift/Obj-C) expuesto a JS.
- **Puente JS ↔ Nativo**: implementado como **TurboModule** (preferido, +puntos) o **NativeModule clásico** (aceptable si se justifica). Explica la diferencia y por qué elegiste uno.
- **Tipado en TypeScript** de la API pública de la librería.
- **Pruebas unitarias propias** de la librería (obligatorias) y su **agent/skill** de IA.
- **README de la librería** con: instalación, `autolinking`, API, ejemplo de uso y cómo se probaría/publicaría.
- **Consumida por la capa `mobile/`** del examen (la app importa la librería como dependencia, **no** copia el código).

Qué debe contener la librería (propuesta)

La librería debe exponer **al menos un componente/módulo con UI y algo nativo**. Elige **una** de estas opciones (o propón una equivalente y justifícala):

- **Opción A — Componente de aporte (input + botón)**: un componente `<DepositInput onConfirm={...} />` con un **input** de monto y un **botón** "Abonar". El botón, al presionarse, invoca **código nativo** (validación/formateo de moneda nativo, o `Toast` /haptic feedback nativo) y devuelve el valor a JS. Se usa en el detalle de la meta.
- **Opción B — Diálogo nativo de confirmación**: un método `showConfirmDialog({ title, message })` que abra un **diálogo/alert NATIVO** (`AlertDialog` en Android / `UIAlertController` en iOS, **no** el `Alert` de RN) y resuelva una `Promise<boolean>` con la elección del usuario. Se usa para confirmar el abono o avisar la meta cumplida (HU 4).
- **Opción C — Notificación local nativa**: `notifyGoalCompleted(goalName: string)` que dispare una **notificación local** / `Toast` **nativo** cuando la meta llega al 100%.

Mínimo exigido: **un input, un botón o un diálogo** con **punto real a código nativo**. Combinar varios (ej. input + botón que abre diálogo nativo) suma valor. Una librería pequeña pero **bien empaquetada, probada y consumida** vale más que una grande a medio integrar.

7. Arquitectura, DDD y patrones de diseño

Debes **proponer y justificar** una arquitectura. No hay una única respuesta correcta; evaluamos coherencia y criterio.

- **DDD**: separa explícitamente las capas:
 - **Domain** (entidades/ `value objects`: `SavingsGoal`, `Money`, `Progress`; reglas de negocio puras y testeables, sin dependencias de RN).
 - **Application** (casos de uso: `MakeDeposit`, `GetGoals`).
 - **Infrastructure** (repositorios, WebView bridge, Redux, módulo nativo/librería).
 - **Presentation** (pantallas/componentes/hooks).
 - **Al menos un patrón de diseño** aplicado y **nombrado** en el código y el README. Ejemplos válidos:
 - **Adapter** para traducir `postMessage` crudo → eventos de dominio.
 - **Repository** para el acceso a metas.
 - **Observer/Pub-Sub** para el canal WebView ↔ nativo.
 - **Factory / Strategy** para construir handlers de mensajes.
 - Las reglas de negocio del dominio deben ser **puras** (funciones testeables sin montar la app).
-

8. Sustentación (30 min)

- ~15 min: demo en vivo + explicación de la arquitectura y decisiones.
 - ~5 min: uso de IA (skills/agents) y qué corregiste de lo generado.
 - ~10 min: preguntas técnicas
-

9. Restricciones y notas

- **Expo prohibido**. Proyecto creado con la CLI oficial de la comunidad.
- No se requiere backend real ni credenciales; usa mocks/datos en memoria. **No** subas secretos, tokens ni PII al repo público.
- **Redux es obligatorio** como fuente de verdad global.

- Si no alcanzas a completar algo, **documenta en el README qué falta y cómo lo harías** — eso también puntúa.
-

10. Checklist de entrega

- [] Repositorio **público** en GitHub con historial incremental.
 - [] **3 componentes**: `web/`, `libreria/`, `mobile/` (monorepo preferido; 2–3 repos aceptados).
 - [] Capa mobile creada con la **CLI oficial** (sin Expo), RN 0.81+ (o última estable).
 - [] `web/`: micro-app que solo emite/recibe `postMessage` (sin tests, no evaluada).
 - [] Pantalla nativa de listado (Redux) + WebView de detalle/abono.
 - [] `postMessage` **bidireccional** con contrato tipado.
 - [] Estado global en **Redux** actualizado desde la web.
 - [] `libreria/`: **librería custom con componente nativo**, con **tests** y **agent/skill**, consumida por `mobile/`.
 - [] `mobile/`: capa final con **tests** y **agent/skill**.
 - [] **Pruebas unitarias** con coverage del core en `libreria/` y `mobile/`.
 - [] Código en **TypeScript** (core tipado).
 - [] **Skill + Agent** de IA + `docs/ia/USO_IA.md` en las capas evaluadas.
 - [] **README** raíz completo (estructura, setup, arquitectura, contrato `postMessage`, uso de IA).
-

Éxitos. Recuerda: preferimos ver **criterio de ingeniería y decisiones justificadas** sobre una feature 100% terminada.